CS 486/686
Large Language Models & Systems

Yuntian Deng

Lecture 22

Pretraining · alignment · agents & RAG

Learning goals

- Write the **autoregressive** LM objective and next-token loss.
- Explain **tokenization**, **pretraining**, and **scaling laws**.
- Describe **fine-tuning** and **RLHF** / **DPO** alignment.
- Explain **in-context learning** and **decoding**.
- Describe LLM **systems**: agents, tool use, and RAG.

What is a large language model?

> A **decoder-only transformer** (L21), stacked N blocks deep and trained to do one thing: **predict the next token**.

Everything else — writing code, answering questions, translating — emerges from doing that one task extremely well, at scale.

Step 0: tokenization (BPE)

Text is first split into **subword tokens** from a fixed vocabulary (~50k). **Byte-Pair Encoding** starts from characters and repeatedly merges the most frequent pair.

"unbelievable" →

`un` `bel` `iev` `able`

- Common words = one token; rare words split into pieces.
- Each token maps to an id, then to an embedding vector (the X rows from L21).
- The model predicts a distribution over the whole vocabulary at each step.

The training objective: predict the next token

Factor the probability of a sequence left to right, and train to maximize it — i.e. minimize **next-token cross-entropy**:

$$p(x_1, \ldots, x_T) = \prod_{t=1}^{T} p(x_t \mid x_{<t})$$

$$\mathcal{L} = -\sum_{t=1}^{T} \log p(x_t \mid x_{<t})$$

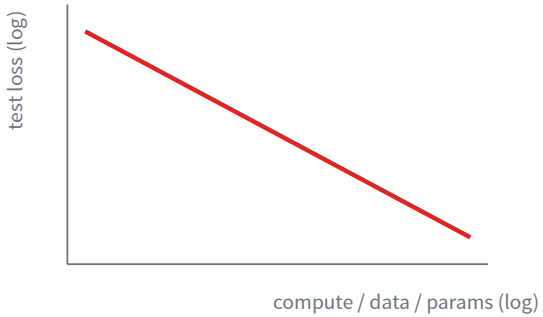
The text *is* the label — no human annotation needed (self-supervised).

Pretraining: one task, enormous scale

- Train on **trillions of tokens** of text (web, books, code).
- Self-supervised → no labels to collect; just predict the next token.
- The result is a **base model**: a broad next-token predictor, not yet a helpful assistant.

Why does simply scaling this up work so well?

Scaling laws



Test loss falls as a smooth **power law** as we scale model size, data, and compute together.

- Predictable: you can forecast a bigger model's loss before training it.
- **Chinchilla:** for a compute budget, balance parameters *and* tokens (don't just grow the model).
- Scaling also unlocks **emergent** abilities not present in small models.

From base model to assistant: fine-tuning

A base model completes text; it doesn't reliably *follow instructions*. **Supervised fine-tuning (SFT)** continues training on curated pairs:

`instruction tuning`

Train on $\{(\text{instruction},\ \text{ideal response})\}$ with the same next-token loss — now the "answer" is a good, on-task response written by humans.

Teaches format and helpfulness, but human-written answers are scarce and don't capture *preferences* between two decent replies.

Alignment: RLHF (and DPO)

Collect human **preferences** (which of two responses is better), fit a **reward model** r, then optimize the policy π — this is the RL from L15, applied to text:

$$\max_{\pi} \; \mathbb{E}_{y \sim \pi}\big[\, r(x, y) \,\big] \;-\; \beta \, \mathrm{KL}\big(\pi \,\|\, \pi_{\mathrm{ref}}\big)$$

- Reward r = "how much humans prefer this answer."
- The **KL** term keeps π close to the fine-tuned model, so it doesn't game the reward.
- **DPO** reaches the same goal by optimizing preferences directly — no separate reward model or RL loop.

In-context learning & prompting

A trained LLM can learn a task from **examples in the prompt** — with *no weight updates*:

```
tiny → small
huge → big
rapid → fast  ← model completes
```

- **Zero-/few-shot:** describe the task (and optionally give examples) in plain language.
- The pattern is inferred at *inference time* from the context window.
- This flexibility is itself an **emergent** ability of large models.

Decoding: turning probabilities into text

At each step the model gives a distribution over the vocabulary. How do we pick the next token?

Greedy / beam

Take the most likely token. Safe but repetitive.

Temperature

Divide logits by T before softmax: higher T = more random, lower = sharper.

Top-k

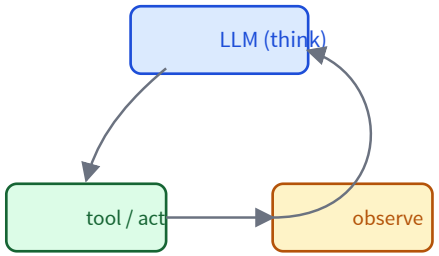
Sample only from the k most likely tokens.

Top-p (nucleus)

Sample from the smallest set whose probability exceeds p.

LLM systems: agents, tools, and RAG

On their own, LLMs can't look things up or take actions. We wrap them in a loop and give them **tools**.



- **Agents:** think → act (call a tool) → observe → repeat.
- **Tools:** web search, a calculator, code execution, APIs.
- **RAG:** retrieve relevant documents and put them in the prompt, so answers are *grounded* in real sources.

Limitations to keep in mind

Hallucination

Fluent text is not truth — models state wrong facts confidently.

Bias & safety

Models absorb biases from training data; alignment is imperfect.

Context & cost

Finite context window; training and serving are expensive.

Evaluation

Hard to measure "good" open-ended answers reliably.

RAG, tools, and human oversight mitigate — but don't eliminate — these.

Learning goals (recap) — Next: generative AI

✓ The **autoregressive** objective and next-token loss.

✓ **Tokenization**, **pretraining**, and **scaling laws**.

✓ **Fine-tuning** and **RLHF / DPO** alignment.

✓ **In-context learning**, **decoding**, and LLM **systems** (agents, RAG).

LLMs generate *text*. L23: generating **images & more** with diffusion.